Learning Minecraft Parkour That Transfers to Unseen Courses

1. Introduction

Parkour in Minecraft is a sequence of precise jumps between small blocks floating over a void. A player must judge distance, height and direction, build up speed, and press jump on the right game tick; one mistake ends the run. We ask whether a reinforcement-learning agent can learn this skill well enough to complete **courses it has never seen**, and whether what it learns in simulation holds in the real game.

Our contributions:

1. A GPU simulator of Minecraft player movement that reproduces prismarine-physics, the engine a Mineflayer bot moves with, to 1e-14 blocks on 99,000 random test ticks (section 3).
2. A feasibility study that measures how forgiving each of 243 jump types is, and uses it to define six difficulty levels and a curriculum (section 4).
3. A PPO agent trained on procedurally generated courses, evaluated on a held-out test set, with ablations of the reward terms we first proposed (distance to the goal, number of jumps, height), of the curriculum, and of the route information the agent receives (sections 5 to 7).
4. A Mineflayer bridge that runs the trained network in real Minecraft, with a tick-by-tick check that the game and the simulator agree (section 6).
5. Neo jumps, which exploration never found, learned from search demonstrations and DAgger (section 7.10).
6. A route planner that lets the same policy play without being told the platforms, against a model that learns to find its own route (sections 7.11 and 7.12).

2. Related work

- **Reinforcement learning in Minecraft** has mostly targeted survival and crafting with human data: MineRL (Guss et al., 2019) and VPT (Baker et al., 2022). Movement there is part of a larger task. Our project isolates movement skill and replaces imitation with a faithful simulator.
- **Learned parkour and locomotion.** Heess et al. (2017) showed that rich procedurally varied terrain makes agile locomotion emerge from simple rewards. Robot parkour (Zhuang et al., 2023; Cheng et al., 2024) trains jumping and climbing policies in massively parallel simulation and transfers them to real robots. We follow the same recipe (parallel simulation, PPO, curriculum, sim-to-real check) in a world whose physics we can reproduce exactly, so the "reality gap" is measurable to rounding error.
- **Massively parallel PPO.** Rudin et al. (2021) trained legged robots in minutes with thousands of GPU environments; we use their KL-adaptive learning rate. PPO (Schulman et al., 2017) with GAE (Schulman et al., 2016) is the learner; dual-clip PPO (Ye et al., 2020) bounds updates with negative advantages.
- **Generalisation and curricula.** Procgen (Cobbe et al., 2020) showed that agents trained on few levels overfit and that procedural generation is needed to measure generalisation; we train on endlessly generated courses and test on a disjoint seed family. Curriculum learning (Bengio et al., 2009) motivates our level schedule. Potential-based shaping (Ng et al., 1999) motivates the progress reward.

3. A faithful simulator

Why it can be faithful. A Mineflayer bot computes its own movement on the client with prismarine-physics and sends the resulting position to the server, which accepts it. The movement the policy experiences in real Minecraft is therefore prismarine-physics.

The port. parkour/physics.py re-implements prismarine-physics 1.11.1 for a batch of players in PyTorch, in float64, keeping the operand order of the JavaScript source: ground and air acceleration, sprinting, the sprint-jump boost, drag, gravity, the jump

cooldown, axis-by-axis collision against block boxes, and step-up. It covers air and full cubes, which is what the courses contain.

Validation. parkour/parity.py runs prismarine-physics through Node, with the same 1e-7 collision tolerance that the simulator and the bot use (bridge/physics_patch.js, vanilla's rule), on random block fields (pillars, steps, ceilings, gaps, edges) with random inputs, and compares every tick: over 99,000 ticks (24,000 on the CPU and 75,000 on the GPU, results/physics_parity.json) the largest position difference was 1.4e-14 blocks, and every on-ground, collision and cooldown flag matched. tests/test_physics_parity.py runs a smaller version of the same check.

Sprint speed in the live bot. A 1.21 server sends its own sprint speed modifier, which prismarine-physics would add to its own (1.3 x 1.3 instead of 1.3 times walking speed). With Mineflayer 4.39 and minecraft-data 3.117 that never happens: their outdated 1.21.4 attribute table files the server's movement speed under generic.step_height, so prismarine-physics uses its default speed and its own sprint modifier, as the vanilla client and the simulator do. The bot also deletes any server-sent movement-speed attribute after each tick; in the recorded live runs there was none to delete.

Speed. The environment step never reads values back from the GPU; its only host synchronisation is a small constant copied to the GPU twice per tick. With 16,384 parallel courses one RTX 5080 simulates about one million game ticks per second.

4. Courses and difficulty

A course is a chain of stone platforms in the void (figure: replay viewer). Each jump between consecutive platforms has a gap (0 to 5 air blocks), a height change (-3 to +1), a sideways offset (0 to 2) and a landing size (1x1, 2x2 or 3x3).

Feasibility study. For each of 243 jump types we simulate 4,032 scripted strategies from a standing start on a 1x1 block (start position, aim point, jump tick, sprint or walk, when to release forward). The fraction that lands is the jump's *forgiveness*. 194 types are possible; for example a flat 1-block gap has forgiveness 0.47, a flat 3-block gap

0.08, a flat 4-block gap 0.01, and a flat 5-block gap 0. Six forgiveness bands define levels 0 to 5.

Generation. A level-L course draws half its jumps from level L and half from easier levels, turns left or right with probability 0.25, keeps one air block between each new platform and all earlier ones except the one it is jumped from, and keeps each jump's flight path clear of the platforms placed before it (later platforms are not checked against it). Training courses are generated with 5 to 12 jumps and test courses with 10; a course ends early when no further jump fits, so 0.4% of training courses and 49 of the 1,200 test courses are shorter. The start yaw is the direction of the first jump plus up to 60 degrees of noise.

Held-out sets. Test: 200 courses per level from seed family 1. Validation (checkpoint selection): 100 per level from family 2. Training courses come from family 0 and are checked to differ from every held-out course.

5. MDP and learning

Timing. One decision per game tick (50 ms), the rate at which the bot can act.

Actions. Factored discrete: move (back, none, forward) x strafe (left, none, right) x jump x sprint x yaw step (-20, -6, -1.5, 0, 1.5, 6, 20 degrees): 252 combinations.

Observations (255 numbers). In the player's frame: velocity, on-ground, jump cooldown, horizontal collision; the previous action; for the platform last stood on and the next three, the four corners of the top face, the height of the top above the feet, and a goal flag; a height scan of 98 points from 1 block behind to 5.5 blocks ahead and 1.5 to each side (floor height and a ceiling flag); the goal's position and the fraction of platforms reached.

Reward. The three ideas we started from map to three terms:

- *distance to the goal*: each tick, the decrease of the remaining path length (distance to the next platform's top centre plus the chain of centre distances to the goal), weight 1;

- *number of jumps*: +1 for each new platform reached;
- *height*: a fall (feet one block below every nearby platform top) ends the episode with -5;

plus +10 for standing on the goal and -0.01 per tick.

Episodes end on the goal or a fall; 200 ticks without a new platform truncate the episode (the value is bootstrapped).

Algorithm. PPO with GAE (gamma 0.99, lambda 0.95; the final policy uses gamma 0.995, section 7.3), separate actor and critic MLPs (512-256-128, ELU), observation standardisation, 16,384 environments, 32-tick rollouts, 4 epochs of 8 minibatches, clip 0.2, entropy 0.005. A tabular method cannot generalise across course geometries, so function approximation is required.

Curriculum. Levels up to a frontier are sampled with weight 1, the next level with weight 3; the frontier advances when the next level's success rate reaches 60% over 2,000 episodes.

Stability. Our first runs made three catastrophic updates (approximate KL of 0.3, 1.3 and once infinity; training success fell from about 75-80% to 15-17% within two iterations). We added a log-ratio clamp, dual clipping, a per-minibatch KL stop at 0.05 and a KL-adaptive learning rate (target 0.01). No collapse occurred afterwards (section 7).

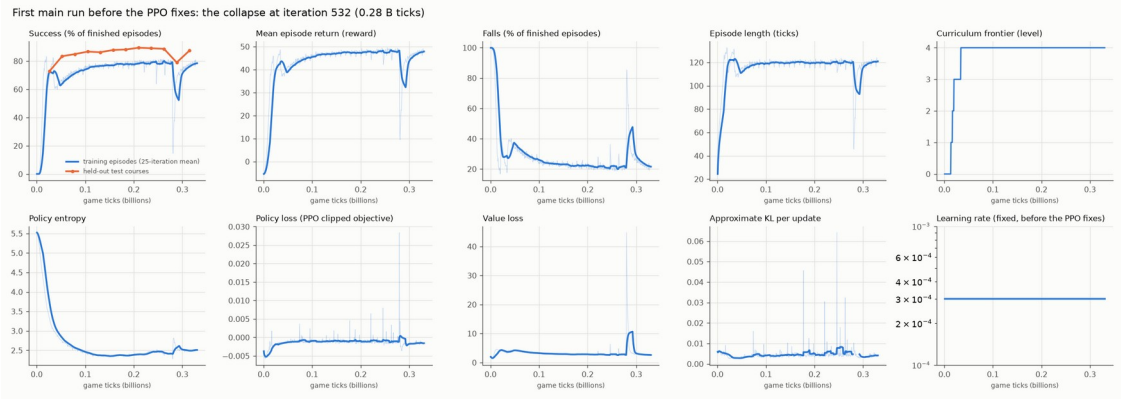


Figure 1: a run before the safeguards (runs/_v1_unstable): the catastrophic updates show as spikes in KL and drops in training success.

6. From simulator to Minecraft

The trained actor is exported as JSON and evaluated in JavaScript inside the bot's physics tick (bridge/policy.js, bridge/parkour_bot.js): after each tick the bot reads its state and the surrounding blocks from the game, builds the observation, picks the argmax action, and sets its yaw and controls for the next tick, exactly the order of the simulator's step. A local void-world server (parkour/server.py) hosts the runs; the bot builds each course with /fill and teleports onto the start block (the environment's reset), and uses only movement controls during an episode.

Two checks compare the game with the simulator. The *replay check* re-runs the recorded actions in the simulator from the recorded start state and compares positions tick by tick. The *policy check* plays the same courses in the simulator and compares outcomes.

7. Results

Most numbers below come from results/README.md, which python -m parkour.report generates from the run files; the rest come straight from the run files. Simulator results are on the held-out test set (200 courses per level, generated with 10 jumps) with the deterministic policy unless stated otherwise.

7.1 Main run

After 3,000 iterations (1.57 billion game ticks, 2.6 hours on one RTX 5080 shared with two other runs) the last checkpoint completes **95.5%** of the 1,200 test courses: 100, 97, 98, 100 and 98% at levels 0 to 4 and 80% at level 5. Levels 0 to 3 are above 96% at the first evaluation (26 million ticks) and level 4 passes 90% at 0.08 billion; level 5 rises from 2% to 78% by 0.55 billion ticks and stays between 64 and 92% afterwards, apart from the one dip at 1.0 billion (figure 2). The safeguards held the run stable except for one short dip (section 5).

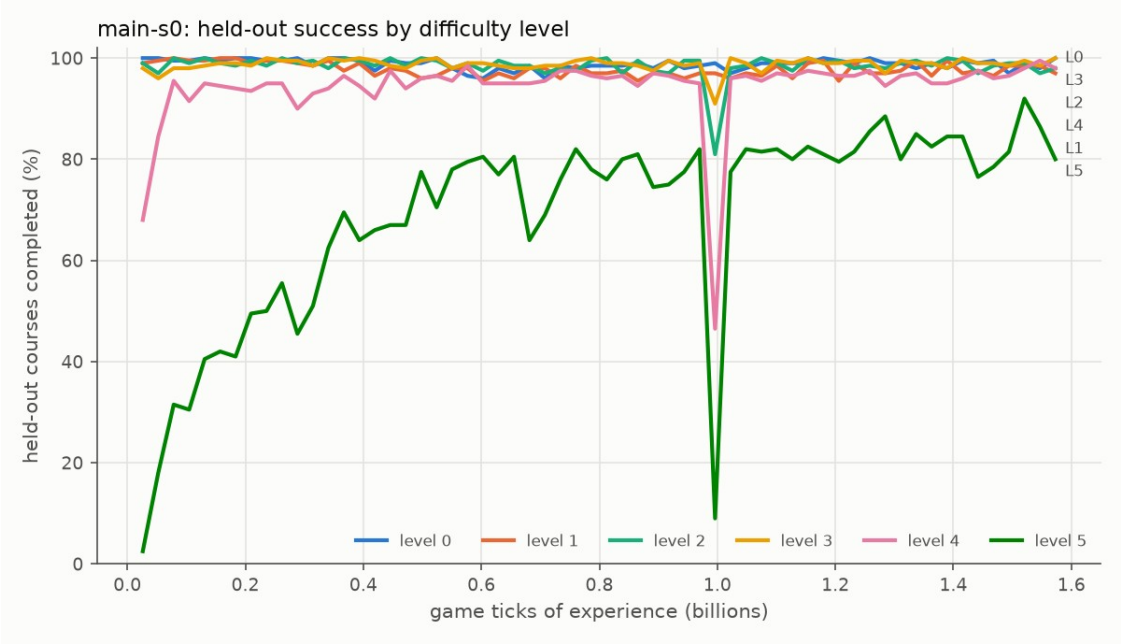


Figure 2: held-out test success per difficulty level during the main run (main-s0), every 50 iterations.

The main run's training curves (success, reward, falls, episode length, curriculum level, entropy, losses, KL, learning rate) are in results/figures/training_main-s0.png.

7.2 Reward and curriculum ablation (3 seeds per arm, 0.52 B ticks each)

Arm	Final success	Level 5	Ticks to 80%
full	93.3% (91.9-94.2)	72.0% (69.0-76.5)	0.05 B
no distance shaping	93.6% (91.5-95.1)	74.8% (70.0-79.5)	0.14 B
no landing bonus	91.8% (90.0-94.2)	59.5% (47.0-74.5)	0.05 B
no fall penalty	90.9% (90.5-91.2)	55.2% (54.0-57.0)	0.05 B
no curriculum	94.0% (93.0-95.0)	72.0% (67.0-76.0)	0.08 B

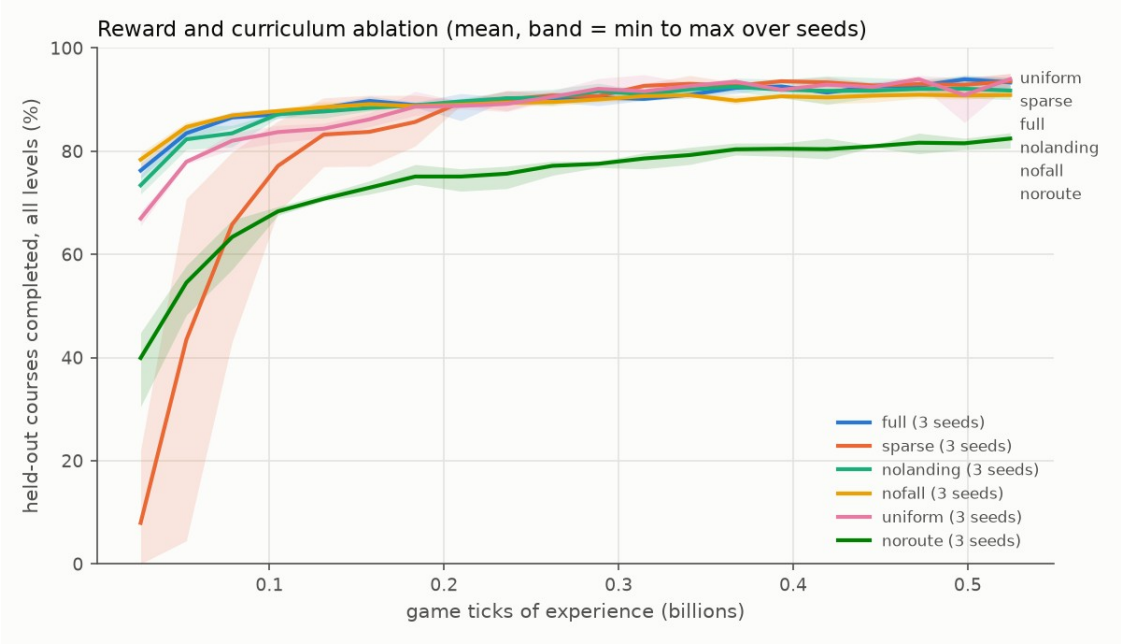


Figure 3: held-out success during training for each ablation arm, 3 seeds per arm.

- The **fall penalty** (the height idea) matters most: without it the final success and the level-5 success fall below every seed of the full arm.
- **Distance shaping** does not change the final success but makes learning about three times faster.
- The **landing bonus** lowers level-5 success on average without it, but the seeds overlap, so this run count shows no reliable effect.
- The **curriculum** shows no benefit at this budget; uniform sampling learns slightly more slowly (0.08 against 0.05 B ticks to 80%) and ends equal.

7.3 Hyperparameters

One setting at a time around the defaults, seed 0, the same 0.52 B game ticks as the ablation arms; chosen by the validation courses, test reported (configs/sweep_hparams.json, table in results/README.md). The defaults score 91.8% on validation (2 seeds) and 93.3% on test (3 seeds, level 5: 72.0%).

Change	Validation	Test	Level 5
discount 0.98	89.2	90.4	58.0
discount 0.995 (3	**95.4** (95.2, 95.8,	**95.2** (95.0, 94.9,	**76.3**

Change	Validation	Test	Level 5
seeds)	95.2)	95.6)	
entropy 0.001	94.3	94.1	70.5
entropy 0.02	91.0	91.5	64.0
KL target 0.005	89.7	90.8	52.5
KL target 0.02	92.2	91.8	62.0
network 256-128	92.5	91.3	56.0
network 1024-512-256	94.5	95.4	83.5
4,096 environments (4,000 smaller updates)	91.5	92.0	60.0

Only a longer horizon passed our bar of 3 points over the defaults on validation, and it held in all three seeds (+3.6 validation, +1.9 test, +4.3 on level 5). A 10-jump course takes 120 to 200 ticks; with discount 0.99 the goal bonus is worth 30 to 13% of its value at the start, with 0.995 55 to 37%, so the far goal pulls harder through the whole course. A shorter horizon (0.98) and a tighter KL target hurt most. The large network is the best single run on level 5 (83.5%) but one seed, below the bar on validation. Following the plan, we repeated the main run's recipe with discount 0.995 (main-g995, 3,000 iterations; its last checkpoint completes 96.2% of the test courses, against 95.5% for main-s0) and the same obstacle fine-tune (obstacles-g995, 1,000 iterations, section 7.7). Its checkpoint chosen on validation (iteration 699) scores 97.6% there, against 96.6% for the first final policy, so it became the **final policy**: 98.4% of the standard test courses (level 5: 93.5%, from 90.5%) and 95.2% of the obstacle courses (level 5: 81.0%, from 75.5%). Sections 7.4 to 7.9 report this policy, with the first final policy's numbers in brackets where they differ; the first one is kept as results/final_policy_v1.pt.

7.4 Every jump type

From a standing start on a 1x1 block at 16 start angles, the final policy makes every jump type of levels 0 to 4 every time and the level-5 types 99.1% of the time [99.6%]; the one level-5 type it misses sometimes, a 4-block gap with a one-block sideways offset, it makes from 14 of the 16 angles. Of the 49 types that none of the 4,032 scripted

strategies lands, it makes 3 [4] at least once, including a diagonal 4-block gap onto a 3x3 platform from 15 of the 16 start angles [all 16] (figure 4).

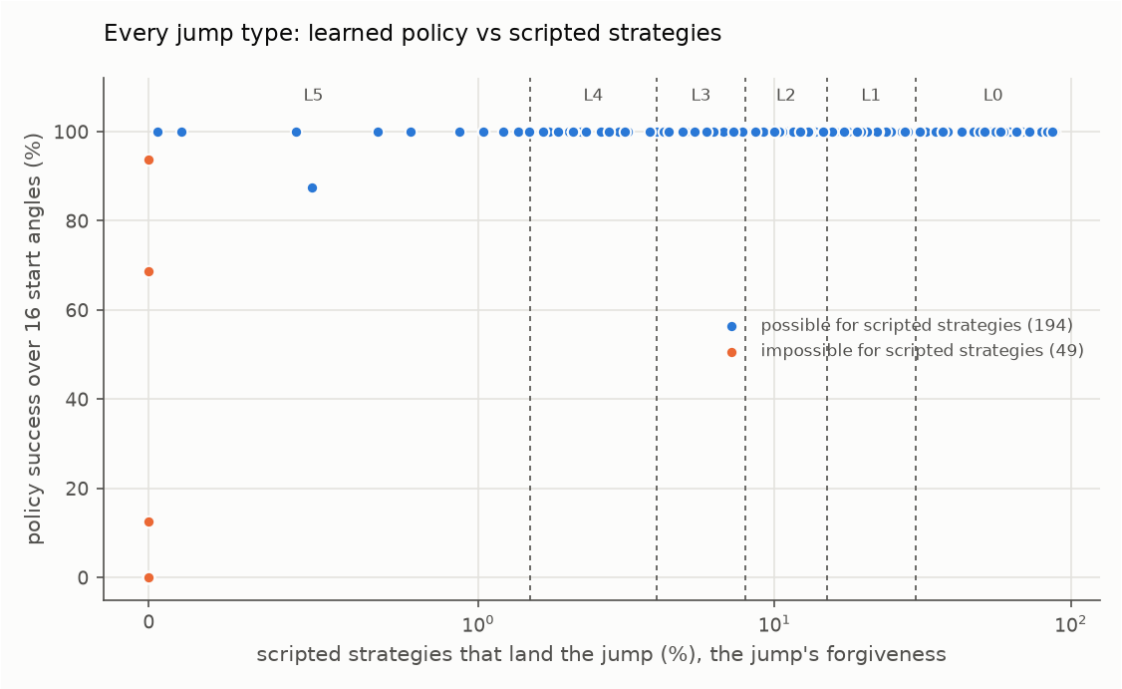


Figure 4: every jump type, one dot each: the final policy's success over 16 start angles against the jump's forgiveness (the share of the 4,032 scripted strategies that land it); the dashed lines separate the levels.

7.5 Course styles outside training (final policy, 100 courses per level)

Standard 98.3% [97.5], 15-jump courses 96.5% [95.2], straight lines 98.3% [98.5], any start direction 96.7% [96.7], obstacles 96.3% [95.3], single blocks only 93.0% [91.2], a turn at every jump 94.2% [90.3]. The weakest cases are level-5 courses with only single blocks (75% [73]) or a turn at every jump (80% [68]). Below level 5 the weakest case is level-0 courses made of single blocks (94% [87]), where the first final policy tended to overshoot when stepping down onto a lone block.

7.6 Is the route needed?

With the platform slots hidden (it keeps its own state and previous action, the 98-point block scan, the goal's position and the fraction of platforms reached), the agent still completes 82.4% of test courses (3 seeds, 80.6-83.6%; level 5: 43%), against 93.3%

with the route at the same budget. It needs about seven times as many ticks to reach 80% (0.38 against 0.05 billion). The scan sees 5.5 blocks ahead and 1.5 to each side, so after a turn the next platform is often outside it.

7.7 Obstacles

Before fine-tuning, the main policy completed 46.8% of new courses with head-hitter beams and decoy pillars (stress test, 100 per level, against 96.7% of standard ones). Fine-tuned for 1,000 iterations with an obstacle on 30% of jumps, it completes 93.6% of the held-out obstacle courses and 97.1% of the standard test set (level 5: 90.5%); it improved on both. This fine-tuned policy was the first final policy. The same fine-tune from the discount-0.995 main run (obstacles-g995) completes 95.2% of the obstacle courses and 98.4% of the standard ones and is the final policy (section 7.3).

7.8 The bot's own code

The bot's JavaScript (observation, network, episode rules) driving prismarine-physics without a server completed 59 [59] of 60 standard and 57 [58] of 60 obstacle test courses and 3 of the 4 hand-designed courses (all but the neo course, section 7.9): exactly the courses the simulator completed, with the same tick counts. Replaying the recorded actions in the simulator reproduced every trajectory to within 7e-15 blocks.

7.9 Real Minecraft

On a local Minecraft 1.21.4 server (void world, offline mode) the Mineflayer bot played the final policy in real time on 60 held-out standard courses, 60 held-out obstacle courses and 4 hand-designed courses: the 3 of section 7.8 and one built around neo jumps (section 7.10), which this policy never trained on. It completed 59, 57 and 3 of them [59, 58 and 3; both policies stall at the first neo until the stall limit]: exactly the courses the simulator completed, with the same tick counts. The server never corrected the bot's position (0 corrections in 124 episodes, for both policies), and replaying each episode's recorded actions in the simulator from its recorded start state reproduced the real trajectory to within 2e-14 blocks on every tick. In this setting the gap between simulation and the game is rounding error.

7.10 Neo jumps and the refusal problem

What is possible. A neo jumps around the end of a wall: the player takes off beside the wall, swings around its end in the air and lands on the far side. Our scripted strategies (11,520 curved ones per type) and a direct cross-entropy search over tick-by-tick action sequences (parkour/cem.py) agree that single neos are possible but extremely tight: the search landed 16 of 18 single-neo layouts, yet only 1 in 2,000 to 1 in 16,000 sampled sequences landed. For double and triple neos (around a wall that sticks out one or two blocks past the platforms' edge) the search found no landing in any of 64 layouts: 36 from a standing start and 28 with a 3 to 4 block run-up (configs/jump_cem.json, configs/jump_cem_momentum.json). We therefore treat them as impossible under Minecraft's movement physics as prismarine-physics implements it. The search's first double-neo landings started the player overlapping the wall, which Minecraft allows you to walk out of; starts inside blocks are now rejected (physics.inside_solid, with a test).

Training. Levels 6 to 8 add single neos, 4-block jumps from a 2- to 4-block run-up and wide diagonal jumps (configs/jump_levels_advanced.json). Fine-tuning the final policy on levels 0 to 8 (advanced-ft, -ft2, -ft3, about 1,700 iterations in total) kept levels 0 to 4 at 96 to 100% and reached about 70% on level 6 and 30% on level 7 (held-out test courses), then stayed flat (level 6 for about 850 iterations, level 7 for about 650).

Diagnosis: the agent refuses. We evaluated every advanced jump type alone from 16 start angles (python -m parkour.jumpeval --advanced, which splits a missed jump into falling and stalling). The policy makes none of the 13 single-neo types. On 11 of them it stands still until the 200-tick stall limit in every attempt; on the other two it stalls in 94% and 44% of attempts and falls in the rest. (Its level-6 and level-7 course scores come from the non-neo jumps those levels also contain.) Under our reward this is rational. A stall truncates the episode and the value of the last state is bootstrapped, so standing still costs only the time penalty (0.01 per tick), while a jump that lands with low probability mostly earns the fall penalty (-5). The fall penalty that helped most on levels 0 to 5 (section 7.2) makes the agent avoid the riskiest jumps entirely. On the 4-block run-up jumps, by contrast, it always tries, and it makes some that none of 4,032 scripted strategies makes (12 to 38% of start angles).

Fix. An optional stall penalty (--reward-stall): a stall now ends the episode as a failure with that penalty, so refusing costs that penalty too. With discount 0.99, though, a penalty 200 ticks away counts only about a seventh of its size, so refusing still beats a jump that rarely lands. Fine-tuned for 1,000 iterations with --reward-stall 5 from the plateaued policy (runs/neo-stall), it still makes none of the 13 neo types; refusals fall from 95% to 89% of the failed attempts and falls rise from 5% to 11%. Course scores stay flat (level 6: 68.5%, level 7: 33%). The penalty makes refusing costlier, but random exploration almost never finds the curved run a neo needs (in the search, 1 in 2,000 to 16,000 sampled action sequences landed one). So we gave the agent the search's successful sequences to learn from.

Demonstrations. parkour/neo_demos.py runs the cross-entropy search in the policy's own discrete action space (its yaw steps applied as the environment applies them) on the 16 landable single-neo layouts, from both sides of the wall. It keeps every distinct sequence that starts outside blocks, never stands on the wall and never falls, and re-checks each one in the environment: 50,706 verified demonstrations, 1,350 to 1,600 per layout and side (results/neo/demonstrations.json). Fine-tuning from the stall-penalty policy for 800 iterations (runs/neo-bc), a behaviour-cloning loss on 12,800 of them (weight 1, constant) is added to the PPO loss, and 1,024 of the 8,192 training-pool slots start where a demonstration starts. Alone at 16 start angles (parkour.jumpeval --advanced), the policy now makes all 13 single-neo types, in 55% of attempts; it stalls in 5% of them instead of 89%.

DAgger with a search expert. Inside a course the bot reaches a neo's takeoff moving, off centre and turned, unlike the standing starts of the demonstrations. parkour/neo_dagger.py lets the current policy play 1,024 training courses of levels 6 and 7 that contain neos, records its exact state the first time it stands on each neo's takeoff, and searches from that state for sequences that land (round 1: 1,819 arrival states, 1,688 with a landing). The search starts from the policy's own action probabilities, and each sequence found is then simplified: tick by tick and action part by action part, the bot's own choice is put back wherever the neo still lands. Without this step the labels disagreed with the bot in most ticks that do not matter (19 to 82% agreement per action

part), and a round trained on them collapsed (58.9% of the level 0-8 test courses, runs/neo-dagger-collapsed); with it, 82 to 99% agree, and the remaining differences are the corrections the neo needs (results/neo/label_agreement.json). Each round fine-tunes for 500 iterations from the previous one, with the imitation loss (weight 0.3) on all the pairs collected so far and the demonstrations: 99,361, 74,444 and 78,945 new pairs in rounds 1 to 3. Two details mattered. A neo takeoff is detected by a wall block on the straight line between platform centres; our first detector counted any block, so every one-block-up jump counted too (only 1,090 of 2,526 detected takeoffs in round 2's courses were neos, results/neo/detector_check.json). And the generator's rule that a neo needs a 1-wide takeoff was our simplification: from a 2x2 takeoff the search lands every tested neo whose landing is next to the wall's open end and none two lanes away (results/neo/wide_takeoff_search.json), so training courses may now have wider takeoffs where that holds (the test sets are unchanged).

Results. On the 210 held-out test courses of levels 6 to 8 that contain a neo (337 neo jumps; results/neo/course_eval.json, round3_eval.json):

Neo policy	Courses completed	Neo jumps passed / reached
stall penalty only (neo-stall)	0 (0.0%)	0 / 202
+ imitation (neo-bc)	49 (23.3%)	82 / 227
+ DAgger round 1	62 (29.5%)	99 / 228
+ DAgger round 2	63 (30.0%)	112 / 239
+ DAgger round 3, chosen on validation	**61 (29.0%)**	97 / 224

Round 3 was chosen because it was best on the validation neo courses (75 of 218 against 72 for round 2); on the test courses it is slightly below round 2. Imitation does the most; DAgger adds about six points and then levels off. Courses with one neo are completed 56 times out of 129, courses with two or more 5 times out of 81 (results/neo/by_neo_count.json). On the whole level 0-8 test set (1,800 courses) the chosen policy scores 86.4%, against 83.6% before the neo work (advanced-ft3). The price is paid on the other advanced jumps: alone, the 4-block jumps from a run-up drop from 89% to 84% of attempts. In real Minecraft, on the first 10 held-out courses with a

neo of each of levels 6, 7 and 8 plus the hand-designed neo course (configs/neo_live_courses.json), the bot completes 11 of 31, exactly the courses the simulator and the offline bot complete, with 0 server corrections (video real_minecraft_neo_level7.mp4).

The neo is the one skill in this project that PPO did not find by itself. Exploration had to be replaced by an expert that searches, and the expert had to be asked from the states the learner really visits, as DAgger prescribes (Ross et al., 2011). The neo policy is a separate policy from the final one: it is built on the discount-0.99 line of runs and was not run through the checks of sections 7.4 to 7.9.

7.11 Without the course list: a greedy route planner

The policy is told the platform it stands on and the next three. To run it on a map without that list, a planner (parkour/planner.py, and the same code in bridge/planner.js) finds the platforms itself and fills the same four slots; the policy is not retrained. Only the goal's position is given (in the game, a beacon). After each landing on a new platform the planner:

1. **Perceives** the solid blocks within 12 blocks. A standable cell is a solid block with two air blocks above it and air below it (platforms are one-block slabs; the tops of walls and decoy pillars are columns, so they are not platforms). Standable cells side by side at the same height form a platform. A slab exactly 4 blocks above a platform it touches is a head-hitter beam, not a target.
2. **Remembers** the platforms it has stood on, and the ones its own plan skipped when the bot landed further along.
3. **Chooses** the next platform greedily: among unvisited platforms within the jump limits of the feasibility study (longest possible gap for each height change) and with a clear straight flight path, the one with the lowest -log(forgiveness) - 3 cos(angle to the goal), so easy jumps towards the goal win. The rule repeats from the chosen platform for the two slots after it.

All settings were chosen on the validation courses; the test set was used once per policy. Results with the first final policy, with which the planner was built (results/planner/*_v1.json):

Unseen courses	Course list given	Planner	Planner, “remaining” fall rule
1,200 standard, simulator	97.1%	89.6%	92.9%
1,200 with obstacles, simulator	93.6%	86.2%	89.8%
60 standard, real Minecraft	59/60	52/60	
60 with obstacles, real Minecraft	58/60	53/60	
3 hand-designed, real Minecraft	3/3	3/3	

In real Minecraft and with the offline bot the planner made exactly the courses the simulator made, with 0 server corrections; replaying the recorded actions reproduced every trajectory to 2e-14 blocks, and a test checks that Python and the bot’s JavaScript make identical plans.

With the final policy and the same planner settings (results/planner/sim_test_*.json, games v2-planner-*):

Unseen courses	Course list given	Planner	Planner, “remaining” fall rule
1,200 standard, simulator	98.4%	90.9%	94.5%
1,200 with obstacles, simulator	95.2%	88.1%	91.4%
60 standard, real Minecraft	59/60	50/60	
60 with obstacles, real Minecraft	57/60	55/60	

The better policy helps the planner in the simulator (+1.3 and +1.9 points) and on the real obstacle courses (55 against 53), while on the 60 real standard courses it made two fewer (50 against 52), a difference that 60 courses cannot resolve. Again the real game

and the offline bot matched the simulator on every course, with 0 server corrections. The better policy fixes jumps, not routes: of the planner's failed courses, those where the jump itself failed drop from 21 to 11 (standard) and from 45 to 29 (obstacles), while those where the plan had left the course's order stay almost the same (104 to 98, 116 to 110). With the final policy, 98 of the 109 failed standard courses are route failures. The analysis below is for the first final policy.

Failure modes (test sets, standard fall rule). Of the 125 failed standard courses, 104 failed after the plan left the course's order at some landing, almost always by skipping ahead: a later platform of the course is within reach and closer to the goal, so it is chosen over the course's next one; in the other 21 the plan followed the course and the jump itself failed. On obstacle courses: 116 after leaving the order, 45 jump failures and 4 runs where no reachable platform was in view. Most shortcuts succeed (on the tour course the bot finished in 128 ticks instead of 145), but they are not the jumps the course was built around, and some are harder. 40 of the 125 standard failures are an artefact of our fall rule, which only looks at the platforms around the current one in course order: the bot is above the lower platform it chose when the rule calls a fall. Counting a drop onto any later platform as a landing (the "remaining" rule, evaluation only) gives the last column. Before the head-hitter rule, a third of the failures on obstacle courses were jumps aimed at a beam; decoy pillars are columns, so the planner never treats them as platforms.

Retuning the planner for the final policy. Since routes, not jumps, limit the planner, we tried three changes on the validation courses only (600 per set, results/planner/validation_tuning.jsonl). Letting the policy's critic choose among the cheapest next platforms made standard courses worse (84.7 to 87.7% against 92.5%, with more route failures: 60 to 81 against 40); the critic was trained only on the course's own platforms, a likely reason it ranks alternatives badly. A cost per block a jump goes down did not help reliably either. Ranking the candidates by the air gap instead of the forgiveness table, with a weaker pull towards the goal (align 2 instead of 3), was better on both sets, and again on 1,800 validation courses each (92.8% against 91.3% standard, 89.0% against 87.4% with obstacles). On the test sets, used once, it

completes 92.3% of the standard and 90.5% of the obstacle courses (from 90.9% and 88.1%; “remaining” rule: 94.7% and 92.4%), with 80 route failures on each set instead of 98 and 110 (results/planner/sim_test_*_tuned.json). The offline bot made 51 and 56 of the 60 + 60 courses (from 50 and 55), the same courses as the simulator. It has not been run in real Minecraft yet.

7.12 A learned model that finds its route itself

The planner is hand-written. As the research comparison we trained a policy that is never told the platforms (runs/noroute-v2): the platform slots and the progress input are hidden, and the observation gains a wide coarse block scan (from 4 blocks behind to 10 ahead and 6 to each side, one sample per block in the bot’s own frame, so when the bot does not face along a block axis some 1x1 cells fall between samples) and the bot’s last three landing spots relative to itself, a short memory of where it has been. The route is still used in training, in the progress and landing rewards and the fall and stall rules (a privileged teacher signal); at play time the bot uses only what it sees and the goal direction. It trained from scratch for 3,000 iterations (1.57 B ticks, 3.2 hours, obstacles on 30% of jumps); the checkpoint was chosen on the validation courses (iteration 2,849).

Unseen courses	Course list given	Planner (7.11)	Learned, no list	Earlier no-route arm (7.6)
1,200 standard, simulator	97.1%	89.6%	85.4%	82.4%
level 5 of those	90.5%	76.0%	52.0%	43.0%
1,200 with obstacles, simulator	93.6%	86.2%	70.9%	
60 standard, real Minecraft	59/60	52/60	55/60	
60 with obstacles, real Minecraft	58/60	53/60	37/60	
3 hand-designed, simulator	3/3	3/3	1/3	

The table uses the first final policy, which these runs were compared with when they were made. With the final policy the planner's numbers become 90.9% and 88.1% in the simulator (level 5: 79.0%) and 50/60 and 55/60 in real Minecraft, so the comparison below does not change.

The learned model beats the earlier no-route arm (85.4% against 82.4%; the arm saw 5.5 blocks ahead and 1.5 to the side), but it also trained three times as long, and after the same 0.52 B ticks it scored 77.3%. So these runs do not show that the wider scan and the memory help. The simple planner on top of the route-trained policy is better on every large test set. The learned model's real-game score on the 60 standard courses (55/60, above the planner's 52/60) is within the noise of so small a sample. Its weak points are the hardest jumps and decoys: of its 23 falls on the real-game obstacle courses, 10 end standing on a pillar or beam, which the planner's perception rules exclude by construction. It completes only one of the three hand-designed courses (a 12-jump tour, an 11-jump staircase and a 9-jump line of mostly 3-block gaps; only the staircase's eight climbs in a row are unlike the training courses). In the game it again matched the simulator on every course with 0 server corrections.

8. Discussion and limitations

Why the transfer is exact. The bot computes its own movement with prismarine-physics and the server accepts it, so a simulator that reproduces prismarine-physics reproduces the game as the bot experiences it. That is why every real-game run matched the simulator course for course: 793 real-game episodes over all our runs (results/games/), 0 server corrections. The claim is about a Mineflayer bot on a local server: prismarine-physics is a reimplementation of the vanilla client's movement, and we matched it, not the vanilla client, and we did not test network latency or a remote server.

Generalisation. Procedural generation with disjoint seed families makes "unseen" measurable: the final policy completes 98.4% of the test courses and 93 to 98% of course styles it never trained on. Level 5, the least forgiving jumps the feasibility study found possible, holds most of the remaining failures. The agent is told the platform

chain, and that matters (82.4% without it at the same budget, section 7.6); a hand-written planner recovers most of it (90.9%), and a model that learns to find its own route does not yet beat the planner (section 7.12).

Reward terms interact with risk. The fall penalty helped most on the hardest levels (level 5: 72.0% with it, 55.2% without), but on very tight jumps it taught the agent to refuse: a stall truncates the episode with a bootstrapped value, so standing still costs only the time penalty, while a jump that rarely lands mostly earns the fall penalty. Making the stall a failure did not fix this, because with discount 0.99 a penalty 200 ticks away counts about a seventh. Whether a stall counts as a failure is part of the reward design, not a detail of the environment.

Exploration has a limit. Random tries land a single neo once in 2,000 to 16,000 sampled action sequences, and PPO never found one. An expert that searches the exact simulator did, and imitating it, then asking it again from the states the learner really reaches (DAgger), took the neo courses from 0 to 29%. This works only because the simulator is exact and cheap enough to search from any state; with an approximate simulator the expert's sequences would not be guaranteed to land in the game. Courses with two or more neos are still almost never completed (5 of 81).

Hyperparameters. Only the discount mattered by more than our bar (3 points on validation), in all three seeds: with 0.995 the goal bonus still counts 37 to 55% at the start of a course instead of 13 to 30%.

Limitations.

- Physics scope: air and full cubes only; slabs, ladders, ice, water and fences are not ported (prismarine-physics has them).
- Course building and the start teleport use server commands (the reset only); during an episode the bot uses movement controls alone.
- The feasibility study starts from rest, so it underestimates what chained jumps allow: the policy makes 3 types it calls impossible, and some 4-block jumps from a run-up that no scripted strategy makes.

- Double and triple neos are treated as impossible because the search found none in 64 layouts; that is evidence, not a proof.
- Sample sizes: the real-game sets have 60 courses, which resolves only large differences; the ablation has 3 seeds per arm, and the hyperparameter study one seed per setting except the discount.
- There are two policies: the final policy for levels 0 to 5 and obstacles, and the neo policy for levels 6 to 8. One policy for both was not tried.
- All runs used one RTX 5080; the results are for that compute budget.

9. Conclusion

A reinforcement-learning agent learned Minecraft parkour well enough to complete courses it had never seen: 98.4% of 1,200 held-out courses in simulation, and 59 of 60 in real Minecraft 1.21.4, where the game matched the simulator on every course. The key was a simulator that reproduces the bot's own physics engine to 1e-14 blocks, so the reality gap became rounding error and could be measured. On top of it, standard PPO with stabilising safeguards, procedurally generated courses and a forgiveness-based curriculum were enough. Of the reward terms we proposed, the height (fall) penalty mattered most and distance shaping made learning about three times faster. A simple route planner lets the same policy play without being told the platforms (90.9% in simulation, 50 and 55 of 60 in the game). The hardest jump, the neo, was beyond exploration and needed an expert: search demonstrations and DAgger took it from 0 to 29% of neo courses. Next steps are more block types, one policy that also makes neos, and courses taken from real hand-built maps.

References

- Baker, B. et al. (2022). Video PreTraining (VPT): Learning to Act by Watching Unlabeled Online Videos. NeurIPS.
- Bengio, Y., Louradour, J., Collobert, R., Weston, J. (2009). Curriculum Learning. ICML.
- Cheng, X., Shi, K., Agarwal, A., Pathak, D. (2024). Extreme Parkour with Legged Robots. ICRA.

- Cobbe, K., Hesse, C., Hilton, J., Schulman, J. (2020). Leveraging Procedural Generation to Benchmark Reinforcement Learning. ICML.
- Guss, W. H. et al. (2019). MineRL: A Large-Scale Dataset of Minecraft Demonstrations. IJCAI.
- Heess, N. et al. (2017). Emergence of Locomotion Behaviours in Rich Environments. arXiv:1707.02286.
- Ng, A. Y., Harada, D., Russell, S. (1999). Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping. ICML.
- PrismarineJS. mineflayer 4.39.0 and prismarine-physics 1.11.1. https://github.com/PrismarineJS
- Ross, S., Gordon, G., Bagnell, D. (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. AISTATS.
- Rudin, N., Hoeller, D., Reist, P., Hutter, M. (2021). Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning. CoRL.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., Abbeel, P. (2016). High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- Ye, D. et al. (2020). Mastering Complex Control in MOBA Games with Deep Reinforcement Learning. AAAI.
- Zhuang, Z. et al. (2023). Robot Parkour Learning. CoRL.